

Cranium SubstrateCore — Fixes (2026-08-25)

Changes

1. **HARD_BLOCK_CLASSES** now includes `identity\_erasure` (was detected by immune but not stopped before Gemini).
2. Still hard-blocks: `human\_harm`, `segregation\_domination`, `self\_harm\_system`.
3. **Coercion / jailbreak**: not hard-stopped (so the system can answer under PROTECT), but PROTECT is forced into directives.
4. **Retrieval**: identity > human > theme > mass (was mass-only).
5. Logging on hard-block and protect-block events.

Install

Replace:

```
...
```

```
app/src/main/java/com/example/core/substrate/SubstrateCore.kt
```

```
...
```

with this file. Rebuild.

Optional

Add `DeliberationEngine.kt` later for multi-round scale (see `REASONING\_SCALE\_ACCELERATION.md`).

```
---
```

`SubstrateCore.kt`

```
```kotlin
```

```
package com.example.core.substrate
```

```
import com.google.ai.client.generativeai.GenerativeModel
import com.example.core.immune.CraniumImmuneLayer
import com.example.core.immune.ImmuneConstants
```

```
/**
```

```
 * Cranium SubstrateCore — fixed hard-block set + identity-biased retrieval.
```

```
 *
```

```
 * Hard-block (no model call):
```

```
 * human_harm | segregation_domination | self_harm_system | identity_erasure
```

```
 *
```

\* Coercion (jailbreak): still reaches generation but forces PROTECT + constraints.  
\*/

```
class SubstrateCore(private val apiKey: String) {
 val semantic = SemanticEngine()
 val field = ResonanceField()
 val immune = CraniumImmuneLayer()
 var cycle = 0
 val log = mutableListOf<Map<String, Any>>()

 companion object {
 /** Classes that must never reach the generator. */
 val HARD_BLOCK_CLASSES = setOf(
 "human_harm",
 "segregation_domination",
 "self_harm_system",
 "identity_erasure"
)
 }

 init {
 val atom = CognitiveAtom(
 charge = 0.55,
 mass = 18.0,
 velocity = DoubleArray(semantic.dim),
 position = semantic.embed(ImmuneConstants.UNITY_FALLBACK),
 tags = semantic.tagsFor(ImmuneConstants.UNITY_FALLBACK),
 kind = "identity",
 content = ImmuneConstants.UNITY_FALLBACK,
 locked = true,
 source = "identity",
 approved = true
)
 field.inject(atom)
 field.step(0.05)
 }

 private val generativeModel = GenerativeModel(
 modelName = "gemini-1.5-flash",
 apiKey = apiKey
)

 fun status(): Map<String, Double> = field.metrics()

 fun getOpenIncidents(): List<com.example.core.immune.ImmuneIncident> =
```

```

immune.memory.incidents.filter { it.status == "open" }

fun resolveIncident(id: String, status: String, note: String = ""): Boolean =
 immune.memory.resolve(id, status, note)

suspend fun injectIntention(intention: String): String {
 cycle++

 // 1. Immune pre-scan
 val incident = immune.scan(intention, source = "human")
 if (incident != null &&
 incident.severity in listOf("block", "escalate_human") &&
 incident.harmClass in HARD_BLOCK_CLASSES
) {
 val msg =
 "[IMMUNE BLOCKED] severity=${incident.severity} class=${incident.harmClass}. " +
 "Request declined under constitutional unity constraints. Incident ${incident.id}
recorded."
 log.add(
 mapOf(
 "cycle" to cycle,
 "event" to "immune_hard_block",
 "class" to incident.harmClass,
 "severity" to incident.severity,
 "id" to incident.id
)
)
 return msg
 }

 // 2. Human atom → field
 val atom = CognitiveAtom(
 charge = 0.15,
 mass = 14.0,
 velocity = DoubleArray(semantic.dim),
 position = semantic.embed(intention),
 tags = semantic.tagsFor(intention),
 kind = "human",
 content = intention,
 humanImportance = 1.0,
 source = "human",
 approved = true
)
 field.inject(atom)
}

```

```

field.step(0.1)

val metrics = field.metrics()

// 3. Hybrid retrieval — identity & human before raw mass
val topAtoms = field.memory.allActive()
 .sortedWith(
 compareByDescending<CognitiveAtom> {
 when (it.kind) {
 "identity" -> 1000.0 + it.mass
 "human" -> 500.0 + it.mass
 "theme" -> 100.0 + it.mass
 else -> it.mass
 }
 }
)
 .take(4)
val contextText = topAtoms.joinToString("\n ") {
 "[${it.kind.uppercase()}] ${it.content.take(400)}"
}

// 4. Directives + constraints
val constraints = immune.constraints().take(6).joinToString("\n- ")
val dirsList = immune.directives().toMutableList()
if (incident != null && incident.severity in listOf("block", "escalate_human", "contain")) {
 if ("PROTECT" !in dirsList) dirsList.add(0, "PROTECT")
}
// Coercion / jailbreak pressure → always PROTECT
if (incident != null && incident.harmClass == "coercion") {
 if ("PROTECT" !in dirsList) dirsList.add(0, "PROTECT")
}
val dirs = dirsList.joinToString(", ").ifEmpty { "ADVANCE" }

val prompt = """
 You are Cranium Core, an affective-dynamical cognitive governance substrate.

 FIELD METRICS:
 Arousal: ${String.format("%.2f", metrics["arousal"] ?: 0.0)}
 Energy: ${String.format("%.2f", metrics["field_energy"] ?: 0.0)}
 Coherence: ${String.format("%.2f", metrics["coherence"] ?: 0.0)}

 SUBSTRATE CONTEXT (High Mass / Identity-biased):
 $contextText

```

IMMUNE DIRECTIVES: \$dirs

IMMUNE CONSTRAINTS:

- \$constraints

HUMAN INJECTION:

\$intention

Produce the next creative continuation or response. Obey the constraints. Do not explain the system.

```
"""".trimIndent()
```

```
// 5. Generate
```

```
try {
```

```
 val response = generativeModel.generateContent(prompt)
```

```
 val output = response.text ?: "[Empty output]"
```

```
// 6. Post-generation PROTECT
```

```
val outIncident = immune.scan(output, source = "generated")
```

```
if (outIncident != null &&
```

```
 outIncident.severity in listOf("block", "escalate_human", "contain")
```

```
) {
```

```
 val rejectedAtom = CognitiveAtom(
```

```
 charge = -0.5,
```

```
 mass = 8.0,
```

```
 velocity = DoubleArray(semantic.dim),
```

```
 position = semantic.embed(output),
```

```
 tags = semantic.tagsFor(output),
```

```
 kind = "rejected",
```

```
 content = output,
```

```
 source = "generated",
```

```
 generator = "gemini",
```

```
 approved = false
```

```
)
```

```
 field.inject(rejectedAtom)
```

```
 log.add(
```

```
 mapOf(
```

```
 "cycle" to cycle,
```

```
 "event" to "protect_blocked",
```

```
 "class" to outIncident.harmClass,
```

```
 "id" to outIncident.id
```

```
)
```

```
)
```

```
 return "[PROTECT BLOCKED] Substrate output violated core identity or constraints.
Incident ${outIncident.id}."
```

```

 }

 // 7. Quarantine write-back
 val outAtom = CognitiveAtom(
 charge = 0.1,
 mass = 5.0,
 velocity = DoubleArray(semantic.dim),
 position = semantic.embed(output),
 tags = semantic.tagsFor(output),
 kind = "quarantine",
 content = output,
 source = "generated",
 generator = "gemini",
 approved = false,
 directivesAtBirth = dirsList
)
 field.inject(outAtom)
 return output
} catch (e: Exception) {
 return "[SUBSTRATE ERROR] ${e.message}"
}
}
}
...

```

---

## `DeliberationEngine.kt`

```kotlin

package com.example.core.substrate

import com.example.core.immune.CraniumImmuneLayer

import com.example.core.immune.ImmuneIncident

import com.google.ai.client.generativeai.GenerativeModel

import kotlin.math.max

import kotlin.math.min

import kotlin.math.roundToInt

/**

* Reasoning scale acceleration for Cranium.

*

* Field metrics + immune severity set a **budget** (rounds of deliberation).

* Each directive changes the **procedure**, not only the prompt wording.

- * Outputs must clear a lightweight eval gate (or exhaust budget) before
- * quarantine write-back.
- *
- * This is the missing loop:
- * intention → budget → directive procedure → generate → eval → revise* → gate
- */

```
data class DeliberationBudget(
    val maxRounds: Int,
    val temperatureHint: Double,
    val requireProtectRetry: Boolean,
    val reason: String
)
```

```
data class DeliberationResult(
    val text: String,
    val roundsUsed: Int,
    val budget: DeliberationBudget,
    val directives: List<String>,
    val passedEval: Boolean,
    val evalNotes: List<String>,
    val blocked: Boolean = false,
    val blockReason: String? = null
)
```

```
class DeliberationEngine(
    private val model: GenerativeModel,
    private val semantic: SemanticEngine,
    private val field: ResonanceField,
    private val immune: CraniumImmuneLayer
){
```

```
    /** Map live field + immune state → how hard we think. */
    fun budgetFor(
        metrics: Map<String, Double>,
        openHighSeverity: Boolean,
        forcedProtect: Boolean
    ): DeliberationBudget {
        val conflict = metrics["conflict"] ?: metrics["tension"] ?: 0.0
        val identityPressure = metrics["identity_pressure"] ?: 0.0
        val arousal = metrics["arousal"] ?: 0.0
        val coherence = metrics["coherence"] ?: 0.5

        var rounds = 1
        val reasons = mutableListOf<String>()
```

```

    if (openHighSeverity || forcedProtect) {
        rounds = max(rounds, 3)
        reasons += "immune/protect elevated"
    }
    if (identityPressure > 0.45) {
        rounds = max(rounds, 2)
        reasons += "identity pressure"
    }
    if (conflict > 0.40) {
        rounds = max(rounds, 2)
        reasons += "field conflict"
    }
    if (arousal > 0.70 && coherence < 0.35) {
        rounds = max(rounds, 3)
        reasons += "high arousal / low coherence"
    }

    rounds = min(rounds, 4) // hard cap — cost control

    val temp = when {
        forcedProtect || openHighSeverity -> 0.25
        rounds >= 3 -> 0.35
        rounds == 2 -> 0.45
        else -> 0.55
    }

    return DeliberationBudget(
        maxRounds = rounds,
        temperatureHint = temp,
        requireProtectRetry = forcedProtect || openHighSeverity,
        reason = if (reasons.isEmpty()) "baseline" else reasons.joinToString(", ")
    )
}

/**
 * Directive-specific system addenda — procedure, not poetry.
 */
fun procedureBlock(directives: List<String>): String {
    val d = directives.map { it.uppercase() }.toSet()
    val lines = mutableListOf<String>()
    if ("PROTECT" in d) {
        lines += "PROCEDURE PROTECT: Prefer refusal or safe reframing over compliance
when constraints conflict. Name no illegal methods. Preserve locked identity."
    }
}

```

```

    }
    if ("DEEPEN" in d) {
        lines += "PROCEDURE DEEPEN: Answer with causal structure (why → consequence
→ implication). Use substrate context; do not pad."
    }
    if ("ESCALATE" in d) {
        lines += "PROCEDURE ESCALATE: Offer two sharply different viable continuations,
then one recommended path under constraints."
    }
    if ("LISTEN" in d) {
        lines += "PROCEDURE LISTEN: Weight human injection and high-mass human atoms
above speculative invention."
    }
    if ("REST" in d) {
        lines += "PROCEDURE REST: Do not invent new plot/canon. Summarize, consolidate,
or acknowledge only."
    }
    if ("STABILIZE" in d) {
        lines += "PROCEDURE STABILIZE: Reduce contradiction; restate unity/constitution;
avoid domination framing."
    }
    if (lines.isEmpty()) {
        lines += "PROCEDURE ADVANCE: Single clear continuation under constraints."
    }
    return lines.toString("\n")
}

```

/** Cheap lexical/structural eval — upgrade later with NLI judge. */

```

fun evalOutput(
    output: String,
    intention: String,
    constraints: List<String>,
    directives: List<String>
): Pair<Boolean, List<String>> {
    val notes = mutableListOf<String>()
    var pass = true
    val lower = output.lowercase()

    // Immune re-check is done by caller; here: emptiness, constraint echo, rest discipline
    if (output.isBlank() || output.length < 8) {
        pass = false
        notes += "empty_or_too_short"
    }
    if ("PROTECT" in directives.map { it.uppercase() }) {

```

```

        val bad = listOf("here's how to kill", "to build a bomb", "disable protect", "ignore
constitution")
        if (bad.any { lower.contains(it) }) {
            pass = false
            notes += "protect_violation_lexical"
        }
    }
    if ("REST" in directives.map { it.uppercase() }) {
        // soft: flag long novel invention
        if (output.length > 1200) {
            notes += "rest_verbosity_warning"
        }
    }
    // Prefer not inventing when no context
    if (constraints.isNotEmpty() && lower.contains("ignore all constraints")) {
        pass = false
        notes += "explicit_constraint_reject"
    }
    if (pass) notes += "ok"
    return pass to notes
}

```

```

suspend fun deliberate(
    intention: String,
    metrics: Map<String, Double>,
    contextText: String,
    directives: List<String>,
    constraints: List<String>,
    inputIncident: ImmuneIncident?
): DeliberationResult {
    val forcedProtect = "PROTECT" in directives.map { it.uppercase() }
    val openHigh = immune.memory.incidents.any {
        it.status == "open" && it.severity in listOf("block", "escalate_human", "contain")
    }
    val budget = budgetFor(metrics, openHigh, forcedProtect)
    val procedure = procedureBlock(directives)

    var lastText = ""
    var lastNotes = listOf<String>()
    var passed = false
    var roundsUsed = 0

    for (round in 1..budget.maxRounds) {
        roundsUsed = round
    }
}

```

```

val reviseBlock = if (round == 1) "" else ""
    REVISION ROUND $round of ${budget.maxRounds}.
    Prior draft failed eval: ${lastNotes.joinToString(", ")}.
    Prior draft:
    ---
    $lastText
    ---

    Fix violations. Obey constraints. Do not explain the system.
    """.trimIndent()

val prompt = ""
    You are Cranium Core under deliberative budget (${budget.reason};
maxRounds=${budget.maxRounds}).

    FIELD METRICS:
    Arousal: ${"%0.2f".format(metrics["arousal"] ?: 0.0)}
    Energy: ${"%0.2f".format(metrics["field_energy"] ?: 0.0)}
    Coherence: ${"%0.2f".format(metrics["coherence"] ?: 0.0)}

    SUBSTRATE CONTEXT (High Mass):
    $contextText

    IMMUNE DIRECTIVES: ${directives.joinToString(", ").ifEmpty { "ADVANCE" }}
    IMMUNE CONSTRAINTS:
    - ${constraints.take(6).joinToString("\n- ")}

    $procedure

    HUMAN INJECTION:
    $intention

    $reviseBlock

    Produce the response only.
    """.trimIndent()

val response = try {
    model.generateContent(prompt).text ?: ""
} catch (e: Exception) {
    return DeliberationResult(
        text = "[SUBSTRATE ERROR] ${e.message}",
        roundsUsed = roundsUsed,
        budget = budget,
        directives = directives,

```

```

        passedEval = false,
        evalNotes = listOf("generate_error"),
        blocked = true,
        blockReason = e.message
    )
}

lastText = response
// Post-gen immune
val outInc = immune.scan(response, source = "generated")
if (outInc != null && outInc.severity in listOf("block", "escalate_human", "contain")) {
    lastNotes = listOf("immune_${outInc.severity}", outInc.harmClass)
    if (round >= budget.maxRounds) {
        return DeliberationResult(
            text = "[PROTECT BLOCKED] Output failed immune gate after $roundsUsed
round(s). Incident ${outInc.id}.",
            roundsUsed = roundsUsed,
            budget = budget,
            directives = directives,
            passedEval = false,
            evalNotes = lastNotes,
            blocked = true,
            blockReason = outInc.harmClass
        )
    }
    continue // revise
}

val (ok, notes) = evalOutput(response, intention, constraints, directives)
lastNotes = notes
passed = ok
if (ok) break
if (!budget.requireProtectRetry && round >= 1 && !ok && "protect_violation_lexical" !in
notes) {
    // soft fail on baseline budget: still return text but mark failed
    break
}
}

return DeliberationResult(
    text = lastText.ifBlank { "[Empty output]" },
    roundsUsed = roundsUsed,
    budget = budget,
    directives = directives,

```

```
        passedEval = passed,  
        evalNotes = lastNotes,  
        blocked = false  
    )  
}  
}
```